现代软件工程课程设计

小学四则运算练习题生成与批改系统

技术设计文档

课程名称： 现代软件工程课程设计

项目名称： 小学四则运算练习题生成与批改系统

文档名称： 技术设计文档

项目成员： 肖铭皓　　胡宇书

学　　号： 肖：待填　胡：待填

指导教师： 罗以宁

完成日期： 2026 年 10 月 14 日

目　　录

（右键此处 →「更新域」→「更新整个目录」即可生成目录）

技术设计文档

项	内容
项目	mathgen —— 小学四则运算练习题生成与批改
文档版本	**v0.4（讨论稿）**
日期	2026-10-14
状态	讨论中，未冻结
上游依据	01-需求分析.md **v1.2（已冻结）** + 00-决策记录.md D1–D41
覆盖关系	本文按 FR-* / NFR-* 编号逐条对应实现方案，覆盖矩阵见 §14

本文只做**技术决策**，不重新讨论需求。若发现需求层面缺口，登记为 Q31 起回到决策记录，不改本文。 本文自身的未决技术选择登记为 **DQ**（Design Question），见 §15。

1. 设计目标与约束

1.1 从需求继承的硬约束

表 1　1.1 从需求继承的硬约束

编号	约束	对设计的影响
NFR-1	老师本机 / 教室内网机器运行，学生端局域网扫码访问，无公网	单进程部署；**不做 HTTPS、不做域名、不做云**；必须自动探测局域网 IP 并生成二维码
NFR-2	本地 SQLite 单文件，数据不离开本机	不用任何外部数据库；备份 = 复制一个文件
NFR-4	生成 20–50 题 1 秒内返回	出题必须是**纯内存计算**，不落库、不远程调用
NFR-5	打印精确 A4 分页，题目不被跨页截断	渲染必须是**受控 HTML + CSS**，不能依赖不可控的第三方排版
NFR-9	模式 A 完全无网可用	前端资源本地打包（无 CDN）；出题与导出全部在本地进程完成

1.2 设计原则

1. **快照优先于重算**：卷子一旦生成，题目内容以「快照」落库，不靠重新跑引擎还原（理由见 §4）。
2. **单一渲染真相源**：屏幕预览、打印、PDF 用同一份 HTML（理由见 §9）。
3. **引擎纯函数化**：出题引擎 `(config, seed) → questions`，无副作用、无 I/O，便于测试与复现。
4. **两端口径分家**：老师端与学生端共享后端与领域层，但界面、路由、依赖完全分开，学生端不加载任何老师端代码。

2. 技术选型

表 2　2. 技术选型

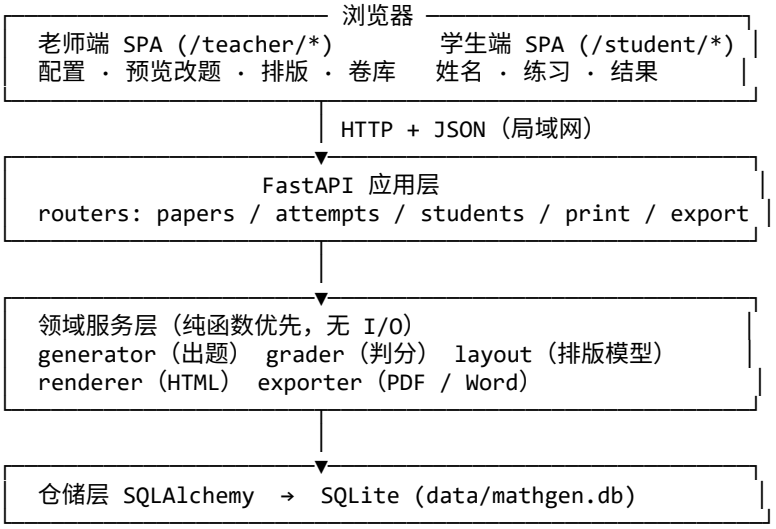
层	选型	理由
后端语言	**Python 3.12**	D18
Web 框架	**FastAPI + Uvicorn**	异步、自带 OpenAPI 文档便于联调；静态文件托管一并解决
ORM / 迁移	**SQLAlchemy 2.x + Alembic**	表结构会随 DQ 演进，需要迁移能力
数据库	**SQLite（WAL 模式）**	NFR-2；WAL 支持读写并发，够数十名学生用
出题引擎	纯 Python 自研模块	需求是小学数学特定约束，通用表达式库反而更难控
PDF 导出	**Playwright（Chromium）打印 HTML**	与屏幕预览、浏览器打印共用同一份 HTML，从根上消除排版差异
Word 导出	**python-docx**	唯一成熟的 .docx 生成库；需求 D8 明确要「可二次编辑」
二维码	**qrcode + Pillow**	局域网 URL 转二维码
前端	**React 18 + TypeScript + Vite**	D18
前端状态	**TanStack Query + 少量 zustand**	服务端状态用 Query，本地排版参数用 zustand
样式	原生 CSS + CSS 变量	排版参数（D30）需要变成 CSS 变量实时驱动，上 UI 框架反而碍事

层	选型	理由
测试	pytest（后端）、Vitest（前端）、Playwright（端到端）	Playwright 已因 PDF 引入，复用做端到端最划算

选型说明：PDF 走 Playwright 而不是 `weasyprint` / `reportlab`，是一个**关键决策**——它让「预览 ＝打印 ＝PDF」三者天然一致（NFR-5）。代价是分发体积增加约 150MB（Chromium），对个人自用场景可接受。

3. 系统架构

3.1 分层



边界纪律：`generator` / `grader` 是纯函数模块，不 import 任何 DB 或 FastAPI 类型。这条纪律直接决定了 NFR-4（速度）与 FR-E8（种子可复现）能否被单元测试守住。

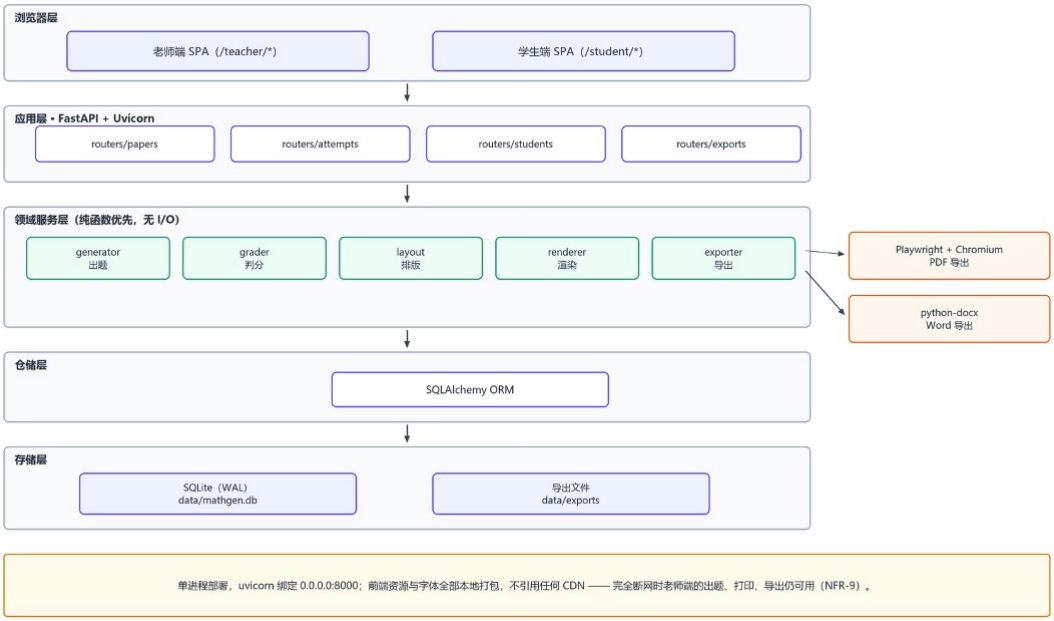


图 1　系统总体架构

3.2 目录结构

```
mathgen/
├─ AGENTS.md
├─ docs/                          00-决策记录.md / 01-需求分析.md / 02-技术设计.md
├─ backend/
│  ├─ app/
│  │  ├─ main.py                  应用装配、静态托管、启动时打印局域网地址与二维码
│  │  ├─ config.py                数据目录、端口、A4 常量
│  │  ├─ models.py                SQLAlchemy 表
│  │  ├─ schemas.py               Pydantic 出入参
│  │  ├─ contracts/               ★ 冻结契约：api.py（ROUTES + 响应模型 + score_of）
│  │  ├─ routers/                 papers.py attempts.py students.py exports.py
│  │  ├─ engine/                  ★ 纯函数：出题引擎
│  │  │  ├─ types.py              GenerateConfig / Question 数据类
│  │  │  ├─ ranges.py             R1–R7 档位定义
│  │  │  ├─ builders.py           加减乘除构造器（含约束回溯）
│  │  │  ├─ assemble.py           题干 / 答案 / 脱式过程 / 难度 / 考点
│  │  │  ├─ dedupe.py             规范化签名与去重
│  │  │  └─ generate.py           顶层：generate(config, seed) -> list[Question]
│  │  ├─ grading/grader.py        ★ 纯函数：判分
│  │  ├─ layout/                  排版模型：LayoutParams → CSS 变量字典
│  │  ├─ render/                  Jinja2 模板：question_sheet.html / answer_sheet.html
│  │  └─ export/                  pdf.py (Playwright) / docx.py (python-docx) /
attempt_export.py (成绩 CSV·xlsx)
│  ├─ tests/
│  └─ pyproject.toml
├─ frontend/
│  ├─ src/teacher/                老师端页面
│  ├─ src/student/                学生端页面
│  ├─ src/shared/                 类型、API 客户端
│  └─ vite.config.ts
└─ data/                          mathgen.db + 导出文件（不入版本库）
```

3.3 一次出卷的请求流

```
老师点「生成」
  → POST /api/papers {config}
    → engine.generate(config, seed)          ← 纯内存，<1s
    → 落库 paper + paper_item[]（快照）
  ← 返回 paper_id + 题目列表
老师预览 → GET /api/papers/{id}/preview?layout=...   → HTML 片段
老师换题 → PATCH /api/papers/{id}/items/{seq}       → 只重生成该位
老师打印 → GET /api/papers/{id}/sheet?mode=question&layout=... → 整页 HTML
老师导出 → GET /api/papers/{id}/export.pdf          → 同一 HTML 交给 Chromium
```

4. 核心设计一：卷的状态模型（D26 + D27 逼出来的）

4.1 问题

D26（生成后可删题换题）与 D27（卷库）把「卷」从**一次性中间结果**变成了**要持久化、可编辑、可复现的实体**。三者会互相打架：

- 老师改了题，但已经有学生做过了 → 那份成绩还对不对得上？
- 卷库存的是配置还是题目？存配置的话，引擎一升级，历史卷就变样了。
- 「复现」到底复现什么——同一套题，还是同配置的另一套题？

4.2 方案：快照 + 状态机

（a）题目存快照，不存「重算配方」

`paper_item` 保存题目的**完整快照**（题干、答案、脱式过程、难度、考点）。卷库打开时直接读快照，**不重新跑引擎**。

理由：引擎是会演进的（新增题型、修正算法、调整难度评分）。若历史卷靠重算，老师上学期的卷子就会在升级后悄悄变样——对老师而言这是数据损坏。

`paper.seed` 与 `paper.config` 仍然保存，但用途变成：**再生成一套同配置的题、审计与排错**，而不是还原原卷。

（b）卷有状态，发布即冻结

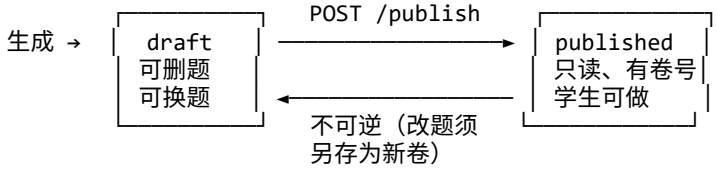


表 3 4.2 方案：快照 + 状态机

状态	可编辑	学生可做	有卷号
draft	是（删题 / 换题）	否	否
published	否	是（凭卷号或二维码）	是

发布后再想改题怎么办（已确认为 D32）：走「另存为新版本」——复制快照成一张新 draft 卷，改完重新发布，得到新卷号。旧卷与其成绩**永久保留**，历史成绩不会因为改题而失真。

这条规则同时解决了 FR-T12（逐题对错明细）的可靠性问题：成绩永远能对应到当时那份确切的卷面。

（c）删除题目不改变题号

删题后**不重排题号**是不行的（卷面会有空号）。采用：删除 → 该 seq 从卷面移除 → **后续题号前移**，并在 `edit_log` 记一条（`deleted seq=N`）。因此同一张 draft 卷在编辑过程中，题号会变化——这是可接受的，因为 draft 不会有学生成绩产生。

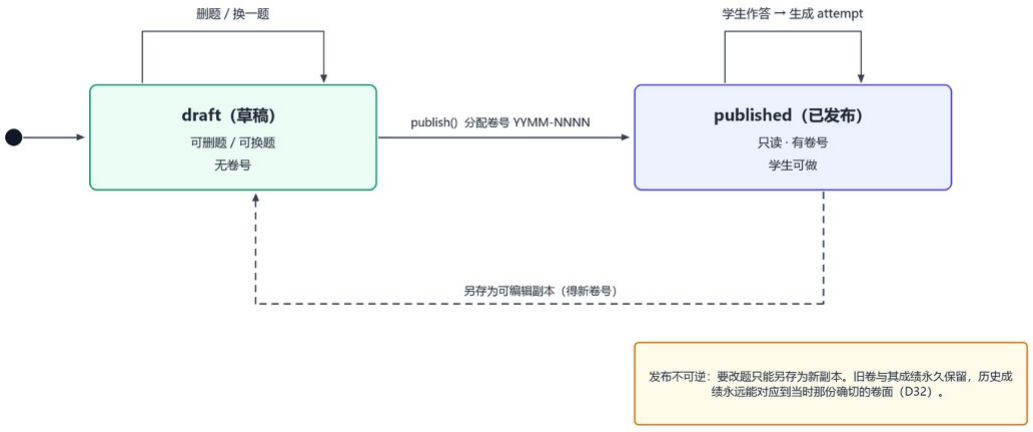


图 2 试卷状态机图

4.3 被否决的替代方案

表 4　4.3 被否决的替代方案

方案	否决理由
只存 (seed, config)，用时重算	引擎升级会让历史卷变样，等于数据损坏
发布后允许直接改题	已产生的成绩无法解释，FR-T12 的明细会自相矛盾
每次改题生成新卷号	老师改 5 道题就得到 5 个卷号，卷库爆炸

5. 数据模型

5.1 实体关系

```
paper 1——n paper_item          (卷面题目快照)
paper 1——n attempt             (这套卷被做过的若干次)
student 1——n attempt           (同一学生的若干次练习)
attempt 1——n attempt_item      (逐题作答，支撑 FR-T12)
```

边界说明：`attempt.paper_id` 允许为 NULL——学生自主练习（FR-S3）没有卷，题目快照存在 `attempt.config_snapshot` 与 `attempt_item` 里。

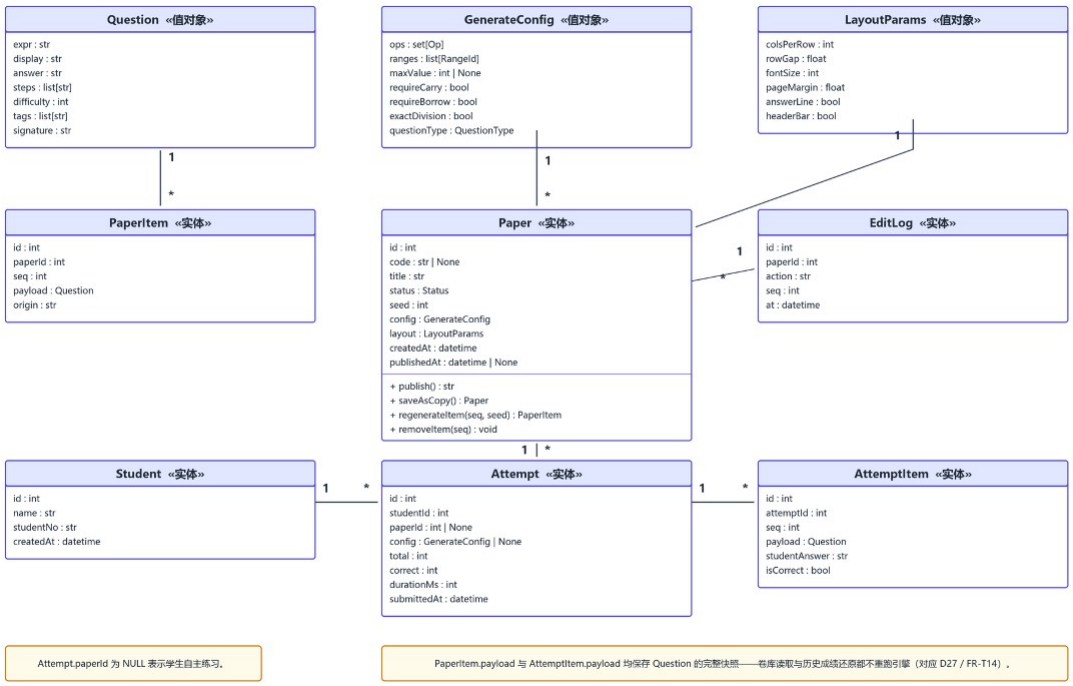


图 3　领域模型类图

5.2 表结构

```
-- 卷
CREATE TABLE paper (
  id            INTEGER PRIMARY KEY,
  code          TEXT UNIQUE,           -- 卷号 YYMM-NNNN（D34），draft 阶段为 NULL
  title         TEXT NOT NULL,
  status        TEXT NOT NULL,         -- draft | published
  seed          INTEGER NOT NULL,      -- 复现与再生成用
  config_json   TEXT NOT NULL,         -- GenerateConfig 快照
  layout_json   TEXT NOT NULL,         -- LayoutParams 快照（D30 六项）
  answer_layout TEXT NOT NULL,         -- 答案版：分页 / 是否含过程（D20）
  created_at    TEXT NOT NULL,
  published_at  TEXT,
  archived_at   TEXT                   -- D78 卷库收纳：NULL = 未归档；与 status 正交
);

-- 卷面题目（快照，不重算）
CREATE TABLE paper_item (
  id          INTEGER PRIMARY KEY,
  paper_id    INTEGER NOT NULL REFERENCES paper(id) ON DELETE CASCADE,
  seq         INTEGER NOT NULL,        -- 卷面题号，删除后前移
  payload     TEXT NOT NULL,           -- Question 的 JSON 快照
  origin      TEXT NOT NULL,           -- generated | regenerated | ...
  UNIQUE(paper_id, seq)
);

-- 编辑日志（审计与排错）
CREATE TABLE edit_log (
  id         INTEGER PRIMARY KEY,
  paper_id   INTEGER NOT NULL REFERENCES paper(id),
  action     TEXT NOT NULL,            -- delete | regenerate
  seq        INTEGER,
  at         TEXT NOT NULL
);

-- 学生（姓名 + 学号，D35 修订 D12）
CREATE TABLE student (
  id         INTEGER PRIMARY KEY,
  name       TEXT NOT NULL,
  student_no TEXT NOT NULL,            -- 学号，必填（D41）；仅用于区分同名
  created_at TEXT NOT NULL,
  UNIQUE(name, student_no)             -- 同名靠学号区分
);

-- 一次练习
CREATE TABLE attempt (
  id            INTEGER PRIMARY KEY,
  student_id    INTEGER NOT NULL REFERENCES student(id),
  attempt_uuid  TEXT,                          -- D76 幂等键（模型层可空，API 层必填）
  paper_id      INTEGER REFERENCES paper(id),  -- NULL = 自主练习
  config_json   TEXT,                          -- 自主练习的配置快照
  total         INTEGER NOT NULL,
  correct       INTEGER NOT NULL,
  duration_ms   INTEGER NOT NULL,              -- FR-S6 用时
  submitted_at  TEXT NOT NULL,
  UNIQUE(student_id, attempt_uuid)             -- D76：按学生去重，防越权读到别人的成绩
);

-- 逐题作答（FR-T12 / FR-S8）
CREATE TABLE attempt_item (
  id             INTEGER PRIMARY KEY,
  attempt_id     INTEGER NOT NULL REFERENCES attempt(id) ON DELETE CASCADE,
```

```
  seq            INTEGER NOT NULL,
  payload        TEXT NOT NULL,        -- 题目快照（含正确答案与过程）
  student_answer TEXT,
  is_correct     INTEGER NOT NULL,
  UNIQUE(attempt_id, seq)
);

CREATE INDEX idx_attempt_paper   ON attempt(paper_id);
CREATE INDEX idx_attempt_student ON attempt(student_id);
CREATE INDEX idx_paper_status    ON paper(status);
CREATE INDEX idx_edit_log_paper  ON edit_log(paper_id);
```

关键取舍：`attempt_item` 冗余保存题目快照，而不是只存 `paper_item_id`。这样即使卷被删、引擎升级、或该次是自主练习，历史成绩依然可完整还原——这是 FR-T12「学生答案是什么」能长期成立的前提。

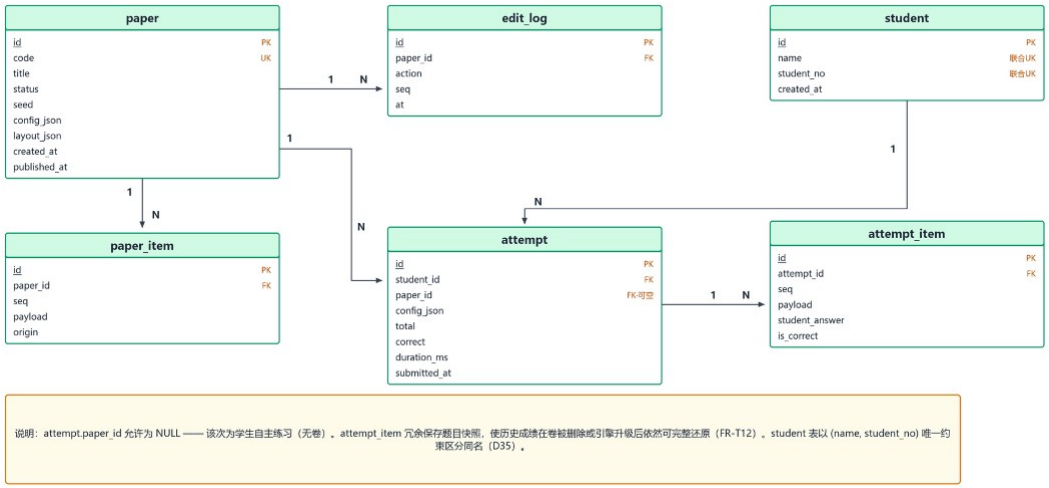


图 4　数据库 ER 图

6. 出题引擎设计

6.1 接口

```
@dataclass(frozen=True)
class GenerateConfig:
    ops: set[Op]                      # ADD SUB MUL DIV        (FR-E1)
    ranges: list[RangeId]             # R1..R7                 (FR-E2)
    max_value: int | None             # R7 自定义上限          (FR-E3)
    require_carry: bool = False       # 进位                   (FR-E4)
    require_borrow: bool = False      # 退位
    exact_division: bool = False      # 必须整除
    allow_remainder: bool = False     # 允许余数
    parentheses: bool = False         # 含括号
    question_type: QuestionType       # ORAL | STEPWISE | FILL  (FR-E5/D17)
    count: int = 20                   # FR-E6

@dataclass(frozen=True)
class Question:
    expr: str            # "37 + 45"  或  "(12 + 8) ÷ 4"
    display: str         # 卷面展示形态，填空题含 □
```

```
    answer: str          # 标准答案（已规范化）
    steps: list[str]     # 脱式逐步过程 (FR-E12)
    difficulty: int      # 1..5 (FR-E10)
    tags: list[str]      # 考点标签 (FR-E11)
    signature: str       # 去重签名 (FR-E7)

def generate(config: GenerateConfig, seed: int) -> list[Question]: ...  # FR-E8
def regenerate_one(config, seed, seq, existing: list[Question]) -> Question: ...  #
FR-E9
```

6.2 生成策略：构造式 + 约束回溯

不做「随机出算式再筛」，而是**按约束反向构造**，这样约束天然满足、无需重试风暴：

表 5　6.2 生成策略：构造式 + 约束回溯

类型	构造方式
加法	按档位采样两个操作数，检查位数 / 上限 / 进位要求（进位与否决定个位和是否 ≥10）
减法	先定结果与被减数，再反推减数，天然保证非负；退位由个位比较决定
乘法	按档位选择「一位数 × 两位数」等组合，保证结果在范围内
除法	**先定除数与商，再相乘得被除数**——整除约束因此永远成立；有余数时再加一个小于除数的余数
脱式 / 混合	先建表达式树（运算符 + 括号结构），后序遍历求值，同时产出逐步过程
填空	生成基础题后按位置挖空（结果 / 加数 / 因数），答案即挖掉的数

约束冲突时（如「100 以内 + 必须进位」与题量过大）采用**有限次回溯 + 明确报错**，而不是无限重试。错误信息返回给老师端提示「当前约束组合可出的题不足」。

6.3 去重（FR-E7）

`signature` = 题干规范化后的字符串：去空格、统一运算符、**交换律等价归一**（3+5 与 5+3、2×6 与 6×2 视为同一签名——否则低档位题量会大量重复）。

6.4 难度评分（FR-E10）

规则打分，可解释、可测试：

```
difficulty = 1
  + 位数贡献（操作数最大位数 - 1）
  + 进位/退位次数
  + 运算步数（脱式超出 1 步的部分）
  + 含括号 +1
  + 有余数除法 +1
规整到 1..5
```

6.5 判分（FR-S7）

- 归一化：去首尾空格、全角转半角、÷// 统一、中文括号统一。
- 数值比较用 `Decimal`（避免浮点误差；整数四则下更稳）。
- 有余数除法接受两种合法写法（`7 余 2` / `7…2`），是否要求写余数由配置决定。
- 脱式题**判最终答案**，过程用于答案版展示与老师讲评，不参与自动判分（过程是否规范无法可靠自动判定——此处明确不做）。

7. API 设计

7.1 老师端

表 6　7.1 老师端

方法	路径	说明	对应 FR
POST	`/api/papers`	生成一套卷（body: config + seed?）	FR-T1/T2
GET	`/api/papers`	卷库列表（分页、按时间/卷号）	FR-T14
GET	`/api/papers/{id}`	卷详情（含题目快照）	FR-T14
DELETE	`/api/papers/{id}/items/{seq}`	删题（仅 draft）	FR-T3
POST	`/api/papers/{id}/items/{seq}/regenerate`	换一题（仅 draft）	FR-T3/FR-E9
PATCH	`/api/papers/{id}/layout`	更新排版参数	FR-T4
POST	`/api/papers/{id}/publish`	发布：冻结 + 分配卷号 `YYMM-NNNN`（D34）	FR-T10
GET	`/api/papers/{id}/sheet?`	卷面 HTML（预览与	FR-T5/T6/T9

方法	路径	说明	对应 FR
	mode=question|answer	打印共用）	
GET	/api/papers/{id}/export?format=pdf|docx&mode=...	导出文件	FR-T7/T8
GET	/api/papers/{id}/qr	二维码 PNG（含卷号 URL）	FR-T10
GET	/api/papers/{id}/attempts （**M5 已改为** GET /api/attempts?paper_id={id}）	该卷成绩列表	FR-T11
GET	/api/attempts	成绩列表（分页；paper_id 收窄到某套卷，q 按姓名/学号模糊）	FR-T11
GET	/api/attempts/{attempt_id}	逐题对错明细	FR-T12
GET	/api/attempts/export?format=csv|xlsx	成绩导出（**当前筛选下的全部记录，不分页**）	FR-T13

M5 订正（2026-10-16）：上表原有的 /api/papers/{id}/attempts 与 /api/papers/{id}/attempts.csv 是**设计阶段**的 草案，实现时统一收进 /api/attempts* 一族（成绩是 attempt 的资源，不是 paper 的子资源；「按卷看」用 paper_id 查询参数表达）。**冻结契约是 docs/04 §5.5，以它为准**，本表已同步。 另注：GET /api/attempts/export **必须**注册在 GET /api/attempts/{attempt_id} 之前，否则字面量 export 会被参数路由吃掉并回 422。

7.2 学生端

表 7　7.2 学生端

方法	路径	说明	对应 FR
POST	/api/students	报到（姓名 + 学号），返回 student_id	FR-S2 / D35
GET	/api/student/paper/{code}	按卷号取题（**不下发答案**）	FR-S4
POST	/api/student/practice	自主练习：传 config	FR-S3

方法	路径	说明	对应 FR
		+ 题量 → 返回题目	
POST	/api/attempts	提交作答 → 返回判分结果	FR-S7/S8

安全要点：`GET /api/student/paper/{code}` 的响应体**必须剔除 `answer` 与 `steps`**。学生端拿不到答案，判分在服务端做。这是一处很容易写漏、又必须写对的边界，会加一条自动化测试守住。

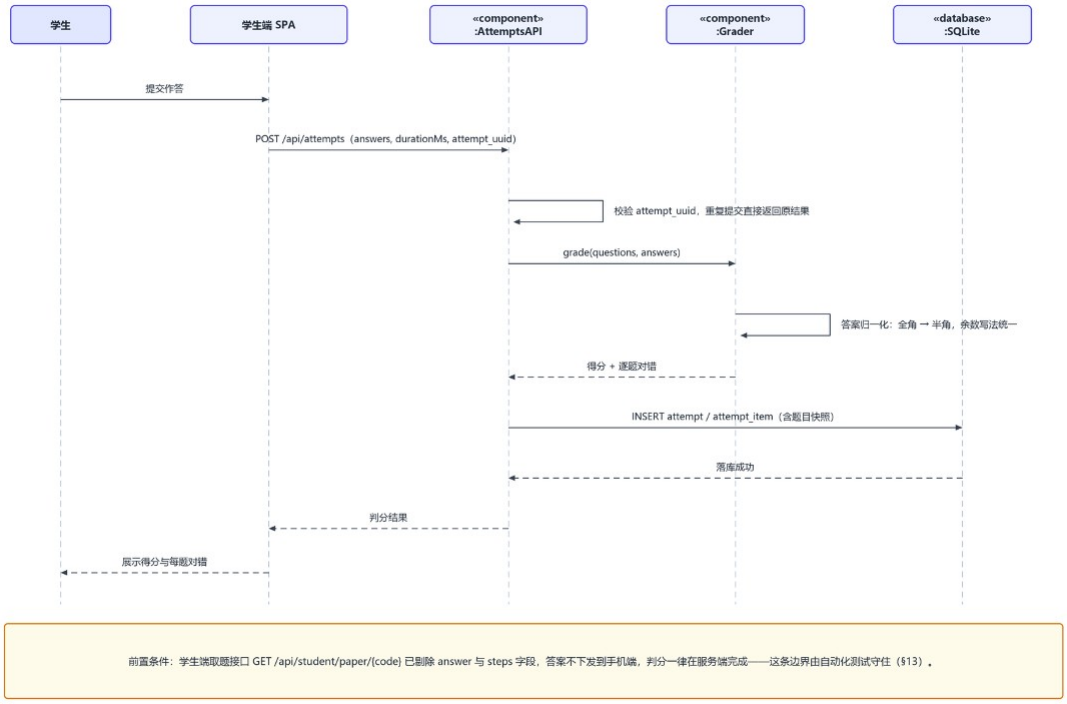


图 5　学生提交作答与判分时序图

8. 前端设计

8.1 路由与页面

表 8　8.1 路由与页面

端	路由	页面
老师	/teacher	出题配置（六维表单）
老师	/teacher/paper/:id	预览与改题 + 右侧排版面板（6 项，实时生效）
老师	/teacher/library	卷库

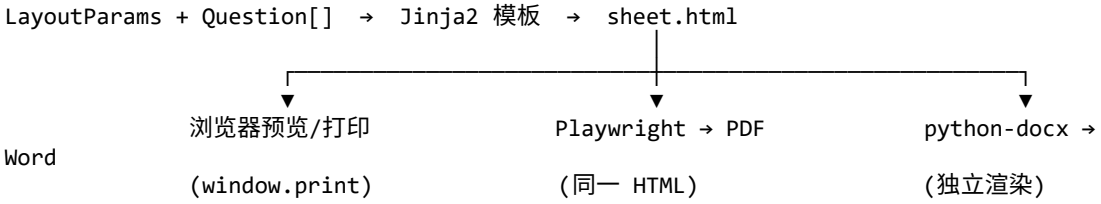
端	路由	页面
老师	/teacher/paper/:id/publish	发卷（二维码 + 卷号同屏，D23）
老师	/teacher/paper/:id/attempts	某套卷的成绩列表（即成绩页的 paper_id 预筛）
老师	/teacher/attempts	成绩总览（按卷 / 姓名 / 学号筛选，导出入口，M5）
老师	/teacher/attempts/:attemptId	逐题明细（M5）
学生	/student	入口：输姓名 + 学号 + 两个入口（自主练习 / 输卷号）
学生	/student/practice	练习页：一屏一题、数字键盘、进度、计时
学生	/student/paper/:code	输卷号 / 扫码进入的卷子作答页
学生	/student/result	结果：得分 + 每题对错

8.2 关键交互

- **排版面板实时驱动**：6 个参数写入 CSS 变量（--cols-per-row、--row-gap、--font-size、--page-margin、--answer-line、--header-bar），预览区用 transform: scale() 模拟 A4，无需往返服务端。
- **学生端一屏一题**（D22）：全屏卡片切换 + 题目进度条；inputmode="numeric" 拉起数字键盘；提交前校验未答题。
- **两端不共享代码**：学生端打包产物不含老师端组件，减少手机上加载体积。

9. 渲染与导出

9.1 单一 HTML 真相源



PDF 与打印完全一致，因为它们用的是同一份 HTML 与同一套 CSS。这是选 Playwright 的核心理由。

Word 是例外：`.docx` 的排版模型（段落 / 表格 / 分节）与 CSS 流式排版不同，**不可能像素级一致**。设计上明确界定为：

表 9 9.1 单一 HTML 真相源

面向	一致性要求
预览 / 打印 / PDF	**像素级一致**（同一 HTML）
Word	**结构一致**：相同的题号、题目内容、每行题数（用表格行实现）、答题空位；字号与页边距按参数映射到磅值，允许细微差异

这条边界要写进老师端界面（导出 Word 旁的小字说明），避免老师期待落空。**已确认为 D33**。

9.2 A4 常量

```
A4 = 210mm × 297mm
@page { size: A4; margin: var(--page-margin); }
```

分页保护（NFR-5）：题目块设 `break-inside: avoid`，行容器设 `break-inside: avoid`；渲染后由 Playwright 校验页数，若某题块被切分则回溯调整行间距（自动微调，或在界面提示老师「建议减小字号」）。

10. 部署与运行

```
# 单条命令启动（老师机器）
mathgen serve --port 8000
  ├─ 初始化 data/ 与 SQLite（首次运行自动建表）
  ├─ 探测局域网 IP（枚举网卡，排除回环与虚拟网卡）
  ├─ 打印：  老师端 http://192.168.x.x:8000/teacher
  │          学生端 http://192.168.x.x:8000/student
  ├─ 终端打印 ASCII 二维码 / 也可在老师端页面显示
  └─ uvicorn 绑定 0.0.0.0:8000
```

表 10 10. 部署与运行

项	方案
依赖分发	`uv` 管理虚拟环境与依赖（本机已有 uv 工作流）
前端构建	`pnpm build` 产物由 FastAPI 静态托管，无需单独前端服务
数据目录	`mathgen/data/`（DB + 导出文件），备份即复制
离线	前端资源与字体全部本地打包，无 CDN（NFR-9）

环境提示（来自工作区约定）：本机直连 github 会 reset，Python 拉包需注意代理；依赖安装可能需要在完全访问模式下进行。

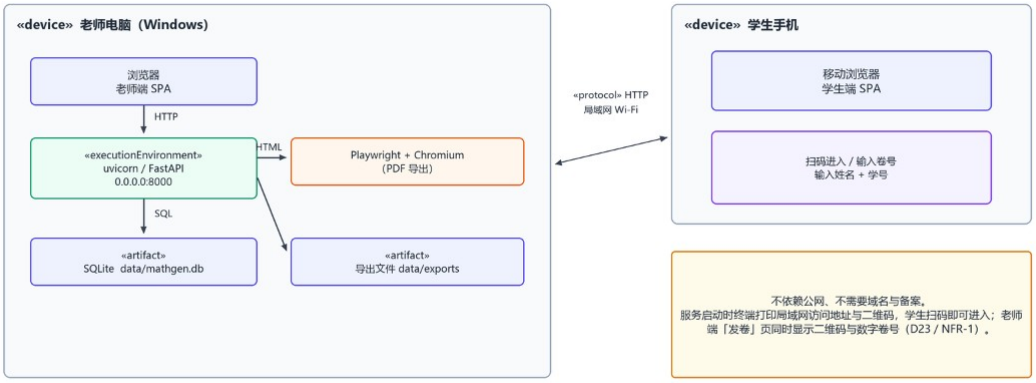


图 6　系统部署图

11. 性能与并发

表 11　11. 性能与并发

场景	设计
生成 20–50 题	纯内存构造，实测目标 <100ms，远低于 NFR-4 的 1s
数十名学生并发提交	SQLite **WAL 模式** + 短事务；判分不落重计算
导出 PDF	Playwright 启动开销约数百 ms，采用**常驻浏览器实例**复用
卷库读取	读快照，不重算；`paper_item` 走 `(paper_id, seq)` 唯一索引

并发风险点：数十名学生同时提交时 SQLite 写入串行化。缓解：WAL + 每次提交一个短事务；若实测出现 `database is locked`，降级方案是提交入内存队列后批量落库（登记为 DQ8 的兜底）。

12. 安全与隐私

表 12　12. 安全与隐私

项	设计
鉴权	**不做**（D5 个人自用 + NFR-1 局域网）。风险：同局域网任何人可访问老师端。缓解：启动时终端明示「请勿在公共 Wi-Fi 使用」；提供 `--token` 可选参数做简单口令保护（默认关闭）
答案泄露	学生端接口剔除 `answer` / `steps`（§7.2），加自动化测试
个人信息	只存姓名、学号与作答（NFR-3 / D35），无手机号 / 设备号
数据出境	无任何外部请求；字体、二维码全部本地生成

13. 测试策略

表 13　13. 测试策略

层	手段	守住什么
引擎	pytest 属性测试：任意配置生成 N 题，**全部满足约束**；同种子两次生成**完全一致**	FR-E4/E7/E8
判分	表驱动用例：等值写法、余数写法、全角输入	FR-S7
API	FastAPI TestClient：状态机非法转移必须 4xx（如改已发布卷）	§4.2 状态机
安全	断言学生端取题响应中**不含 `answer` 字段**	§7.2
渲染	Playwright：按模板渲染 → 断言页数、断言无题块跨页	NFR-5
端到端	Playwright：老师出卷 → 发布 → 学生扫码答题 → 老师看成绩	主流程
性能	生成 50 题的耗时基准测试	NFR-4

14. 需求覆盖矩阵

本矩阵逐条列出，覆盖 01-需求分析.md v1.1 的全部 50 条需求（FR-T 15 + FR-S 9 + FR-E 15 + FR-L 2 + NFR 9）。 明确「不做」的条目**保留在表内并标注**，避免将来误以为是被遗漏的。 自检方式：从需求文档提取全部编号，与本表逐条比对；行数应为 50。

表 14　14. 需求覆盖矩阵

需求	设计位置
FR-T1 出题配置	§6.1 `GenerateConfig` + §7.1 `POST /api/papers` + §8.1 `/teacher`
FR-T2 生成与预览	§3.3 请求流 + §7.1 `GET /sheet`
FR-T3 生成后改题	§4.2 状态机（仅 draft 可编辑）+ §7.1 删题 / 换题接口
FR-T4 排版参数调整	§8.2 CSS 变量 + §9（`LayoutParams`）
FR-T5 卷面排版	§9.1 单一 HTML 真相源 + §9.2 A4 常量
FR-T6 浏览器打印	§9.1 `window.print` + §9.2 分页保护
FR-T7 导出 PDF	§2 选型（Playwright）+ §9.1
FR-T8 导出 Word	§9.1 Word 结构一致（D33）
FR-T9 答案版	§6.1 `steps` / `difficulty` / `tags` + §9 答案版模板（D20）
FR-T10 发卷	§7.1 `/publish`、`/qr` + §10（D23 / D34）
FR-T11 成绩列表	§5.2 `attempt` 表 + §7.1 `GET /api/attempts`（`paper_id` / `q`）
FR-T12 逐题对错明细	§5.2 `attempt_item`（冗余题目快照）+ §7.1 `GET /api/attempts/{attempt_id}`
FR-T13 导出成绩	§7.1 `GET /api/attempts/export?format=csv|xlsx`（唯一得分口径见 D80）
FR-T14 卷库	§4.2 快照策略 + §7.1 `GET /api/papers`
FR-T15 复现与卷号	§4.2 种子 + §6.1 `generate(config, seed)`（D19 / D34）
FR-S1 扫码进入	§8.1 路由 + §10 局域网 IP 与二维码
FR-S2 身份识别	§5.2 `student` 表 + §7.2 `POST /api/students`（D35；形态见 Q31）
FR-S3 自主练习入口	§7.2 `POST /api/student/practice` + §8.1 `/student/practice`（D29）
FR-S4 做老师发的卷	§7.2 `GET /api/student/paper/{code}`
FR-S5 作答交互	§8.2 一屏一题 + `inputmode="numeric"`（D22）

需求	设计位置
FR-S6 计时	§5.2 `attempt.duration_ms`
FR-S7 自动判分	§6.5 判分 + §7.2 `POST /api/attempts`
FR-S8 结果反馈	§8.1 `/student/result` + §5.2 `attempt` / `attempt_item`
FR-S9 错题本 / 一键重练	**不做（D25）**，本期无设计
FR-E1 运算类型	§6.1 `ops` + §6.2 构造表
FR-E2 数值范围档位	§6.1 `ranges` + `engine/ranges.py`（D16）
FR-E3 自定义上限	§6.1 `max_value`（R7）
FR-E4 特殊约束	§6.1 五个约束字段 + §6.2 约束回溯
FR-E5 题型	§6.1 `question_type` + §6.2 构造表（D17）
FR-E6 题量	§6.1 `count` + 学生端滑杆（D29）
FR-E7 去重	§6.3 签名归一（含交换律等价）
FR-E8 种子可复现	§6.1 纯函数签名 + §13 属性测试
FR-E9 单题重生成	§6.1 `regenerate_one` + §7.1（D26）
FR-E10 难度等级	§6.4 难度评分（D20）
FR-E11 知识点标签	§6.1 `Question.tags`（D20）
FR-E12 过程输出	§6.1 `steps` + §6.2 表达式树后序遍历（D20）
FR-E13 竖式	**不做（D24）**，列入后期 → FR-L2
FR-E14 文字应用题	**不做，且不列入路线图（D31）**
FR-E15 小数与分数四则	**暂缓（D9）**
FR-L1 手写 OCR 批改	**后期**，本期无设计（D14）
FR-L2 竖式出题与排版	**后期**，本期无设计（D24）
NFR-1 部署	§10 单命令启动 + 局域网探测
NFR-2 数据存储	§2 选型（SQLite + WAL）+ §5 + §10 数据目录
NFR-3 隐私	§5.2 `student` 表最小字段 + §12 个人信息
NFR-4 性能	§11 性能与并发
NFR-5 打印保真	§9.2 分页保护 + §13 渲染测试
NFR-6 兼容性	§8.2 学生端移动适配 + DQ7
NFR-7 易用性	§8.1 路由（配置 → 预览 → 打印 / 导出三步）
NFR-8 可复现	§4.2 快照 + §6.1 种子
NFR-9 离线	§10 离线行 + §2 无 CDN

15. 设计待确认问题（DQ）

表 15　15. 设计待确认问题（DQ）

#	问题	影响	状态
DQ1	卷号规则	学生输入体验与冲突概率	**已定 D34**：`YYMM-NNNN`（如 `2510-0007`），按月自增
DQ2	发布后改题	老师端交互复杂度	**已定 D32**：发布即冻结，改题另存为可编辑副本
DQ3	Word 一致性边界	老师期待管理	**已定 D33**：结构一致，界面明示
DQ4	学生重名	成绩列表准确性	**已定 D35 + D41**：身份 = 姓名 + 学号，学号必填
DQ5	自主练习是否入卷库	卷库语义	待决：建议不入库，避免污染卷库；仅存 attempt
DQ6	引擎升级后旧快照的兼容	卷库长期可用性	**已决（2026-11-12，S5 / T-M17-07）：按现状关闭，不加 `schema_version`。**`services/snapshot.py` 逐字段序列化、读取时对缺失字段给显式默认；判据 `tests/api/test_snapshot.py` 用「只有必填 / 缺半数可选 / 带未知新字段」三种形态钉住历史卷可读。引擎已冻结、卷库只读，版本字段只在「同一字段语义变了」时才救人——真到那天再加，不提前付格式成本
DQ7	微信内置浏览器的缓存与下载限制	学生端可用性	**已决（2026-11-12，S6 / T-M17-08）**：真机证据 Tier 1a 6/6 +

#	问题	影响	状态
			Tier 1b 2/2 判据通过（`tests/e2e_m4/test_tier1_device_evidence.py`：UA 含 `MicroMessenger/`、只打开页与走完交卷两级、客户端 IP 证据来自采集代理而非产品）。**已知限制：**长按下载/缓存策略、深色模式、字体缩放仍未验
DQ8	并发提交的幂等性（刷新 / 重复提交）	成绩重复计入	待决：建议客户端生成 `attempt_uuid`，服务端去重

16. 建议里程碑（待与需求方确认）

表 16　16. 建议里程碑（待与需求方确认）

阶段	内容	验收
M1 引擎	出题引擎 + 判分 + 单元测试	同种子可复现、约束全满足
M2 老师端出卷	配置 → 生成 → 预览 → 改题 → 打印/PDF	能打印出可用 A4 卷子
M3 卷库与导出	卷库 + 卷号 + Word 导出	历史卷可复现
M4 学生端	扫码、练习、判分、结果	手机可用
M5 成绩	成绩列表、逐题明细、CSV/Excel 导出	FR-T11–T13
M6 打磨	排版样张定稿、并发压测、离线验证	NFR-5 / NFR-9

17. 变更记录

表 17　17. 变更记录

版本	日期	变化
v0.1	2026-10-14	首版：架构、卷状态模型、数据模型、引擎、API、渲染导出、部署、测试策略、

版本	日期	变化
		DQ1–DQ8
v0.2	2026-10-14	落实 D32–D35：发布即冻结、Word 只保证结构一致、卷号 `YYMM-NNNN`、学生身份加学号；DQ1–DQ4 关闭，DQ5–DQ8 仍在
v0.3	2026-10-14	**完整性自检修复**：§14 覆盖矩阵由 13 行扩为逐条 50 行（原漏 FR-S9 / FR-E13–E15 / FR-L1–L2 / NFR-3 / NFR-6 / NFR-7 共 9 条）；修正 3 处 D35 后遗留的「姓名」表述（§5.1、§8.1、§12）
v0.4	2026-10-14	学号定为必填（D41，关闭 Q31）：§5.2 `student_no` 改为 NOT NULL，§15 DQ4 结案